

PROO/SEW III - Assignment: Own Example

Objective

Create a small *JavaFX* application for managing a self-chosen entity. The app must support two separate views - one using a `ListView`, the other a `TableView`. Connect your application to a local database using *JDBC* and follow the *MVC* pattern.

Things To Learn

- Structuring an application using the *Model-View-Controller* pattern.
- Applying *Service Layer* and *Repository*-patterns to separate business logic and data access.
- Accessing and modifying database entries using *JDBC*.
- Binding *observable* lists to both `ListView`s and `TableViews`.
- *Wrapping* observable lists to make them *read-only*, *filterable* and *sortable*.
- Using *Java's Streaming API* to calculate statistics and derive data.

Submission Guidelines

- Your implemented solution as **zipped** *IntelliJ*-project.

Task

You will design and implement a small *JavaFX* application following the *Model-View-Controller (MVC)* pattern. This time, you will create two distinct views - one using a `ListView`, the other a `TableView`. Both will interact with the same data and logic, helping you better understand how the same *model*, *service* and *repository* can power different user interfaces.

Database Schema

Start by designing a small database schema about a topic you're passionate about.

- It can be anything: your favorite games, cars, books, animals, footballers, movies, recipes - the choice is yours.
- Your main entity must have a primary key (*ID*).
- Add at least 3-5 meaningful fields to describe your items.

You are allowed to use *LLMs* to assist you in creating the `CREATE TABLE` statements. Try something like this:

Generate a H2-compatible database schema for one table containing Austrian football players. It should include a unique ID, the name, the position, the club and the number of goals scored. Generate about 20 rows.

Once you're happy with your schema, create the table in your local *H2* database using *IntelliJ*'s database explorer. Make sure to save the creation script and submit it along with your assignment!

Views

Build two separate *FXML* views using *Scene Builder* or manually:

- **ListView.fxml**
 - Displays your items in a **ListView**.
- **TableView.fxml**
 - Displays the same items in a **TableView**.
- Each *view* should also include:
 - Input fields for adding and editing items.
 - Filtering controls, such as **CheckBoxes**, **Sliders** or **ComboBoxes**.
 - Buttons for...
 - * ...adding, updating and deleting items.
 - * ...domain-specific actions (e.g. repaint a car, mark a book as read, score a goal...).

Model

Create a model class that represents your items:

- Include all fields from your database schema.
- Add validation logic if needed (e.g. name must not be empty, rating must be between 1 and 10...).

Service And Repository

Implement both *Service* and *Repository* layers using a *lazy Singleton* implementation.

- The *Repository* should support all *CRUD* operations for interacting with the database.
- The *Service* should store all items in an observable list and provide business logic.
 - Calculate some statistics (e.g. average rating, number of completed tasks...) using *Java Streams*, and exposes them via **Simple...Property** fields.

Controller

Create two controllers to:

- Bind the model's data to the controls in both views.
- Implement actions for buttons and filters.
- Handle errors and feedback using **Alert** dialogs.

Let your creativity guide your project. Just make sure you meet the technical requirements - and have fun with it!